Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	**Slot:** L43 + L44
Regno: 23BCE1145	**Name:** S Vishwajith

EX3 – AES

Code:

Sender:

```
import numpy as np

import socket


def s_box(b):

    s_box = [

        ["63","7c","77","7b","f2","6b","6f","c5","30","01","67","2b","fe","d7","ab","76"],

        ["ca","82","c9","7d","fa","59","47","f0","ad","d4","a2","af","9c","a4","72","c0"],

        ["b7","fd","93","26","36","3f","f7","cc","34","a5","e5","f1","71","d8","31","15"],

        ["04","c7","23","c3","18","96","05","9a","07","12","80","e2","eb","27","b2","75"],

        ["09","83","2c","1a","1b","6e","5a","a0","52","3b","d6","b3","29","e3","2f","84"],

        ["53","d1","00","ed","20","fc","b1","5b","6a","cb","be","39","4a","4c","58","cf"],

        ["d0","ef","aa","fb","43","4d","33","85","45","f9","02","7f","50","3c","9f","a8"],

        ["51","a3","40","8f","92","9d","38","f5","bc","b6","da","21","10","ff","f3","d2"],

        ["cd","0c","13","ec","5f","97","44","17","c4","a7","7e","3d","64","5d","19","73"],

        ["60","81","4f","dc","22","2a","90","88","46","ee","b8","14","de","5e","0b","db"],

        ["e0","32","3a","0a","49","06","24","5c","c2","d3","ac","62","91","95","e4","79"],

        ["e7","c8","37","6d","8d","d5","4e","a9","6c","56","f4","ea","65","7a","ae","08"],

        ["ba","78","25","2e","1c","a6","b4","c6","e8","dd","74","1f","4b","bd","8b","8a"],

        ["70","3e","b5","66","48","03","f6","0e","61","35","57","b9","86","c1","1d","9e"],

        ["e1","f8","98","11","69","d9","8e","94","9b","1e","87","e9","ce","55","28","df"],

        ["8c","a1","89","0d","bf","e6","42","68","41","99","2d","0f","b0","54","bb","16"],

    ]
```

```
    return s_box[int(b[0], 16)][int(b[1], 16)]

def circular_left_shift(s, n):
    return s[n:] + s[:n]

def convert_array(s):
    arr = [s[i:i+2] for i in range(0, len(s), 2)]
    return np.array(arr).reshape(4, 4).T.tolist()

def round_constant(r):
    RC = ["01", "02", "04", "08", "10", "20", "40", "80", "1B", "36"]
    return RC[r]

def g_function(w, r):
    byte = [w[i:i+2] for i in range(0, len(w), 2)]
    byte = circular_left_shift(byte, 1)
    byte = [s_box(b) for b in byte]
    RC = round_constant(r)
    byte[0] = hex(int(RC, 16) ^ int(byte[0], 16))[2:].rjust(2, '0')
    return ''.join(byte)

def get_intermediate_keys(key):
    word = [key[i:i+8] for i in range(0, 32, 8)]
    words = [convert_array(''.join(word))]
    for i in range(10):
        new_word = [None]*4
        w3 = g_function(word[3], i)
        new_word[0] = hex(int(word[0], 16) ^ int(w3, 16))[2:].rjust(8, '0')
        new_word[1] = hex(int(word[1], 16) ^ int(new_word[0], 16))[2:].rjust(8, '0')
        new_word[2] = hex(int(word[2], 16) ^ int(new_word[1], 16))[2:].rjust(8, '0')
        new_word[3] = hex(int(word[3], 16) ^ int(new_word[2], 16))[2:].rjust(8, '0')
```

```
        words.append(convert_array(''.join(new_word.copy())))
        word = new_word.copy()
    return words

def sub_bytes(pt):
    temp = [[None for i in range(4)] for j in range(4)]
    for i in range(4):
        for j in range(4):
            temp[i][j] = s_box(pt[i][j])
    return temp

def shift_rows(pt):
    for i in range(4):
        pt[i] = circular_left_shift(pt[i], i)
    return pt

def gf_mul(a, b):
    A = int(a, 16)
    B = int(b, 16)
    result = 0
    for _ in range(8):
        if B & 1:
            result ^= A
        c = A & 0x80
        A <<= 1
        if c:
            A ^= 0x11B
        A &= 0xFF
        B >>= 1
    return result
```

```
def mix_columns(pt):
    mc = [
        ["02", "03", "01", "01"],
        ["01", "02", "03", "01"],
        ["01", "01", "02", "03"],
        ["03", "01", "01", "02"]
    ]
    temp = [[None]*4 for _ in range(4)]
    for i in range(4):
        for j in range(4):
            temp[i][j] = 0
            for k in range(4):
                temp[i][j] ^= gf_mul(mc[i][k], pt[k][j])
            temp[i][j] = hex(temp[i][j])[2:].rjust(2, '0')
    return temp

def flatten_state(state):
    return ''.join(state[r][c] for c in range(4) for r in range(4))

def AES_Encryption(pt, key):
    PT = convert_array(pt)
    keys = get_intermediate_keys(key)
    temp = [[None for i in range(4)] for j in range(4)]
    for i in range(4):
        for j in range(4):
            temp[i][j] = hex(int(PT[i][j], 16) ^ int(keys[0][i][j], 16))[2:].rjust(2, '0')
    print(f"\nRound 0, converted text: \"{flatten_state(temp)}\", key: \"{flatten_state(keys[0])}\"")
    for k in range(1, 10):
        temp = sub_bytes(temp)
        temp = shift_rows(temp)
        temp = mix_columns(temp)
```

```
        for i in range(4):
            for j in range(4):
                temp[i][j] = hex(int(temp[i][j], 16) ^ int(keys[k][i][j], 16))[2:].rjust(2, '0')
        print(f"Round {k}, converted text: \"{flatten_state(temp)}\", key: \"{flatten_state(keys[k])}\"")
    temp = sub_bytes(temp)
    temp = shift_rows(temp)
    for i in range(4):
        for j in range(4):
            temp[i][j] = hex(int(temp[i][j], 16) ^ int(keys[10][i][j], 16))[2:].rjust(2, '0')
    print(f"Final Round, cipher text: \"{flatten_state(temp)}\", key: \"{flatten_state(keys[10])}\"")
    return flatten_state(temp)

print("AES Sender initialized.\n")
pt = input("Enter a 128-bit plain text in hexadecimal: ")
key = input("Enter a 128-bit key in hexadecimal: ")

ct = AES_Encryption(pt, key)
print(f"\nThe cipher text is: {ct}.")

s = socket.socket()
s.connect(('127.0.0.1', 5995))
s.send(ct.encode().ljust(1024, b'\0'))
s.send(key.encode().ljust(1024, b'\0'))
print("\nThe Cipher text and the key have been sent successfully to the receiver.")
s.close()
```

Receiver:

```
import numpy as np
import socket

def s_box(b):
```

```
    s_box = [
        ["63","7c","77","7b","f2","6b","6f","c5","30","01","67","2b","fe","d7","ab","76"],
        ["ca","82","c9","7d","fa","59","47","f0","ad","d4","a2","af","9c","a4","72","c0"],
        ["b7","fd","93","26","36","3f","f7","cc","34","a5","e5","f1","71","d8","31","15"],
        ["04","c7","23","c3","18","96","05","9a","07","12","80","e2","eb","27","b2","75"],
        ["09","83","2c","1a","1b","6e","5a","a0","52","3b","d6","b3","29","e3","2f","84"],
        ["53","d1","00","ed","20","fc","b1","5b","6a","cb","be","39","4a","4c","58","cf"],
        ["d0","ef","aa","fb","43","4d","33","85","45","f9","02","7f","50","3c","9f","a8"],
        ["51","a3","40","8f","92","9d","38","f5","bc","b6","da","21","10","ff","f3","d2"],
        ["cd","0c","13","ec","5f","97","44","17","c4","a7","7e","3d","64","5d","19","73"],
        ["60","81","4f","dc","22","2a","90","88","46","ee","b8","14","de","5e","0b","db"],
        ["e0","32","3a","0a","49","06","24","5c","c2","d3","ac","62","91","95","e4","79"],
        ["e7","c8","37","6d","8d","d5","4e","a9","6c","56","f4","ea","65","7a","ae","08"],
        ["ba","78","25","2e","1c","a6","b4","c6","e8","dd","74","1f","4b","bd","8b","8a"],
        ["70","3e","b5","66","48","03","f6","0e","61","35","57","b9","86","c1","1d","9e"],
        ["e1","f8","98","11","69","d9","8e","94","9b","1e","87","e9","ce","55","28","df"],
        ["8c","a1","89","0d","bf","e6","42","68","41","99","2d","0f","b0","54","bb","16"],
    ]
    return s_box[int(b[0], 16)][int(b[1], 16)]


def inverse_s_box(b):
    inv_s_box = [
        ["52","09","6a","d5","30","36","a5","38","bf","40","a3","9e","81","f3","d7","fb"],
        ["7c","e3","39","82","9b","2f","ff","87","34","8e","43","44","c4","de","e9","cb"],
        ["54","7b","94","32","a6","c2","23","3d","ee","4c","95","0b","42","fa","c3","4e"],
        ["08","2e","a1","66","28","d9","24","b2","76","5b","a2","49","6d","8b","d1","25"],
        ["72","f8","f6","64","86","68","98","16","d4","a4","5c","cc","5d","65","b6","92"],
        ["6c","70","48","50","fd","ed","b9","da","5e","15","46","57","a7","8d","9d","84"],
        ["90","d8","ab","00","8c","bc","d3","0a","f7","e4","58","05","b8","b3","45","06"],
        ["d0","2c","1e","8f","ca","3f","0f","02","c1","af","bd","03","01","13","8a","6b"],
        ["3a","91","11","41","4f","67","dc","ea","97","f2","cf","ce","f0","b4","e6","73"],
```

```
        ["96","ac","74","22","e7","ad","35","85","e2","f9","37","e8","1c","75","df","6e"],
        ["47","f1","1a","71","1d","29","c5","89","6f","b7","62","0e","aa","18","be","1b"],
        ["fc","56","3e","4b","c6","d2","79","20","9a","db","c0","fe","78","cd","5a","f4"],
        ["1f","dd","a8","33","88","07","c7","31","b1","12","10","59","27","80","ec","5f"],
        ["60","51","7f","a9","19","b5","4a","0d","2d","e5","7a","9f","93","c9","9c","ef"],
        ["a0","e0","3b","4d","ae","2a","f5","b0","c8","eb","bb","3c","83","53","99","61"],
        ["17","2b","04","7e","ba","77","d6","26","e1","69","14","63","55","21","0c","7d"],
    ]
    return inv_s_box[int(b[0], 16)][int(b[1], 16)]

def circular_left_shift(s, n):
    return s[n:] + s[:n]

def circular_right_shift(s, n):
    return s[-n:] + s[:-n]

def convert_array(s):
    arr = [s[i:i+2] for i in range(0, len(s), 2)]
    return np.array(arr).reshape(4, 4).T.tolist()

def round_constant(r):
    RC = ["01", "02", "04", "08", "10", "20", "40", "80", "1B", "36"]
    return RC[r]

def g_function(w, r):
    byte = [w[i:i+2] for i in range(0, len(w), 2)]
    byte = circular_left_shift(byte, 1)
    byte = [s_box(b) for b in byte]
    RC = round_constant(r)
    byte[0] = hex(int(RC, 16) ^ int(byte[0], 16))[2:].rjust(2, '0')
    return ''.join(byte)
```

```
def get_intermediate_keys(key):
    word = [key[i:i+8] for i in range(0, 32, 8)]
    words = [convert_array(''.join(word))]
    for i in range(10):
        new_word = [None]*4
        w3 = g_function(word[3], i)
        new_word[0] = hex(int(word[0], 16) ^ int(w3, 16))[2:].rjust(8, '0')
        new_word[1] = hex(int(word[1], 16) ^ int(new_word[0], 16))[2:].rjust(8, '0')
        new_word[2] = hex(int(word[2], 16) ^ int(new_word[1], 16))[2:].rjust(8, '0')
        new_word[3] = hex(int(word[3], 16) ^ int(new_word[2], 16))[2:].rjust(8, '0')
        words.append(convert_array(''.join(new_word.copy())))
        word = new_word.copy()
    words.reverse()
    return words

def inverse_shift_rows(ct):
    for i in range(4):
        ct[i] = circular_right_shift(ct[i], i)
    return ct

def inverse_sub_bytes(ct):
    temp = [[None]*4 for _ in range(4)]
    for i in range(4):
        for j in range(4):
            temp[i][j] = inverse_s_box(ct[i][j])
    return temp

def flatten_state(state):
    return ''.join(state[r][c] for c in range(4) for r in range(4))
```

```
def gf_mul(a, b):
    A = int(a, 16)
    B = int(b, 16)
    result = 0
    for _ in range(8):
        if B & 1:
            result ^= A
        c = A & 0x80
        A <<= 1
        if c:
            A ^= 0x11B
        A &= 0xFF
        B >>= 1
    return result

def inverse_mix_columns(ct):
    imc = [
        ["0e", "0b", "0d", "09"],
        ["09", "0e", "0b", "0d"],
        ["0d", "09", "0e", "0b"],
        ["0b", "0d", "09", "0e"]
    ]
    temp = [[None]*4 for _ in range(4)]
    for i in range(4):
        for j in range(4):
            temp[i][j] = 0
            for k in range(4):
                temp[i][j] ^= gf_mul(imc[i][k], ct[k][j])
            temp[i][j] = hex(temp[i][j])[2:].rjust(2, '0')
    return temp
```

```
def AES_Decryption(ct, key):
    CT = convert_array(ct)
    keys = get_intermediate_keys(key)
    temp = [[None for i in range(4)] for j in range(4)]
    for i in range(4):
        for j in range(4):
            temp[i][j] = hex(int(CT[i][j], 16) ^ int(keys[0][i][j], 16))[2:].rjust(2, '0')
    temp = inverse_shift_rows(temp)
    temp = inverse_sub_bytes(temp)
    print(f"Round 0, converted text: \"{flatten_state(temp)}\", key: \"{flatten_state(keys[0])}\"")

    for k in range(1, 10):
        for i in range(4):
            for j in range(4):
                temp[i][j] = hex(int(temp[i][j], 16) ^ int(keys[k][i][j], 16))[2:].rjust(2, '0')
        temp = inverse_mix_columns(temp)
        temp = inverse_shift_rows(temp)
        temp = inverse_sub_bytes(temp)
        print(f"Round {k}, converted text: \"{flatten_state(temp)}\", key: \"{flatten_state(keys[k])}\"")
    for i in range(4):
        for j in range(4):
            temp[i][j] = hex(int(temp[i][j], 16) ^ int(keys[10][i][j], 16))[2:].rjust(2, '0')
    print(f"Final Round, plain text: \"{flatten_state(temp)}\", key: \"{flatten_state(keys[10])}\"")
    return flatten_state(temp)

server = socket.socket()
server.bind(('127.0.0.1', 5995))
server.listen(1)
print("AES Receiver initialized and listening for sender.")
conn, addr = server.accept()
print(f"Connection established with sender at {addr}.")
```

```
ct = conn.recv(1024).decode().rstrip('\0')

print(f"\nThe received cipher text is: {ct}.\n")

key = conn.recv(1024).decode().rstrip('\0')

conn.close()

server.close()


pt = AES_Decryption(ct, key)

print(f"\nThe plain text is: {pt}.")
```

Output:

Sender:

```
Windows PowerShell    Windows PowerShell

PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-6 (AES)> python .\sender.py
AES Sender initialized.

Enter a 128-bit plain text in hexadecimal: 43727970746f67726170687900000000
Enter a 128-bit key in hexadecimal: 53656375726974790000000000000000

Round 0, converted text: "10171a050606130b6170687900000000", key: "53656375726974790000000000000000"
Round 1, converted text: "29be8abf666c313aaa90dad245c6dc30", key: "31060016436f746f436f746f436f746f"
Round 2, converted text: "696cfff7909180c0e149d7329f354dc9", key: "9b94a80cd8fbdc639b94a80cd8fbdc63"
Round 3, converted text: "323d4b534ea9b1a63532b0feb89294cc", key: "9012536d48e98f0ed37d27020b86fb61"
Round 4, converted text: "58fa4481601657f9366bdc5f8ed5f91d", key: "dc1dbc4694f433484789144a4c0fef2b"
Round 5, converted text: "8513804381a4ecb2e408ea53f4d14d9e", key: "bac24d6f2e367e2769bf6a6d25b08546"
Round 6, converted text: "1fc9c12ae22ba68450b28c3cde8deff4", key: "7d551750536369773adc031a1f6c865c"
Round 7, converted text: "253b7e3b761b0a6d6290289392f91fb2", key: "6d115d903e7234e704ae37fd1bc2b1a1"
Round 8, converted text: "5fc8ce8b04eaa5efc2b4e6d2f122c173", key: "c8d96f3ff6ab5bd8f2056c25e9c7dd84"
Round 9, converted text: "03c4f567d5f5a37ba1f7c60e31a45b3f", key: "15183021e3b36bf911b607dcf871da58"
Final Round, cipher text: "fba9ee156094081c4003d0644d27e6b6", key: "804f5a6063fc3199724a36458a3bec1d"

The cipher text is: fba9ee156094081c4003d0644d27e6b6.

The Cipher text and the key have been sent successfully to the receiver.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-6 (AES)>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-6 (AES)> python .\receiver.py
AES Receiver initialized and listening for sender.
Connection established with sender at ('127.0.0.1', 65151).

The received cipher text is: fba9ee156094081c4003d0644d27e6b6.

Round 0, converted text: "03c4f567d5f5a37ba1f7c60e31a45b3f", key: "804f5a6063fc3199724a36458a3bec1d"
Round 1, converted text: "5fc8ce8b04eaa5efc2b4e6d2f122c173", key: "15183021e3b36bf911b607dcf871da58"
Round 2, converted text: "253b7e3b761b0a6d6290289392f91fb2", key: "c8d96f3ff6ab5bd8f2056c25e9c7dd84"
Round 3, converted text: "1fc9c12ae22ba68450b28c3cde8deff4", key: "6d115d903e7234e704ae37fd1bc2b1a1"
Round 4, converted text: "8513804381a4ecb2e408ea53f4d14d9e", key: "7d55175053636977 3adc031a1f6c865c"
Round 5, converted text: "58fa4481601657f9366bdc5f8ed5f91d", key: "bac24d6f2e367e2769bf6a6d25b08546"
Round 6, converted text: "323d4b534ea9b1a63532b0feb89294cc", key: "dc1dbc4694f433484789144a4c0fef2b"
Round 7, converted text: "696cfff7909180c0e149d7329f354dc9", key: "9012536d48e98f0ed37d27020b86fb61"
Round 8, converted text: "29be8abf666c313aaa90dad245c6dc30", key: "9b94a80cd8fbdc639b94a80cd8fbdc63"
Round 9, converted text: "10171a050606130b6170687900000000", key: "31060016436f746f436f746f436f746f"
Final Round, plain text: "43727970746f67726170687900000000", key: "53656375726974790000000000000000"

The plain text is: 43727970746f67726170687900000000.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-6 (AES)>
```